

Teoria da Computação

Autômatos Finitos e Linguagens Regulares

Prof. Renê R. Veloso

Definição formal de computação

• Função de transição estendida para AFN

Seja $N = (Q, \Sigma, \delta, q_1, F)$, um autômato, a função de transição estendida de N é a função

$$\hat{\delta}: Q \times \Sigma^* \rightarrow \mathcal{P}(Q)$$

Definida recursivamente da seguinte forma: para $q \in Q$ e $\omega \in \Sigma^*$,

$$\hat{\delta}(q, \omega) = \begin{cases} E(q) & \text{se } \omega = \epsilon \\ E(\delta(\hat{\delta}(q, \alpha), x)) & \text{se } \omega = \alpha x, \text{ com } x \in \Sigma \text{ e } \alpha \in \Sigma^* \end{cases}$$

↙

... é o conjunto de estados ativos em N após computar toda a cadeia ω a partir do estado q .

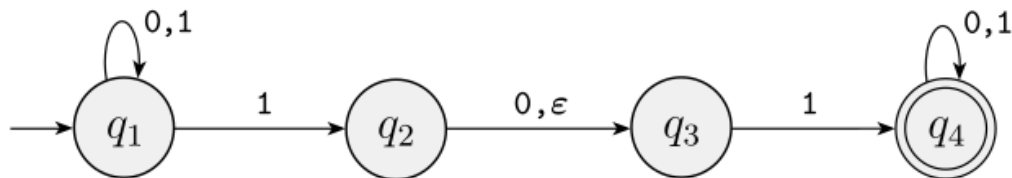
Definição formal de computação

- Aceitação:

Seja $N = (Q, \Sigma, \delta, q_0, F)$ um AFN e seja $\omega \in \Sigma^*$,
dizemos que N aceita ω se $\hat{\delta}(q_0, \omega) \cap F \neq \emptyset$.

Caso contrário, N rejeita ω .

Exemplo



A descrição formal de N_1 é $(Q, \Sigma, \delta, q_1, F)$, onde

1. $Q = \{q_1, q_2, q_3, q_4\}$,
2. $\Sigma = \{0,1\}$,
3. δ é dado como

	0	1	ϵ
q_1	$\{q_1\}$	$\{q_1, q_2\}$	$\emptyset$
q_2	$\{q_3\}$	$\emptyset$	$\{q_3\}$
q_3	$\emptyset$	$\{q_4\}$	$\emptyset$
q_4	$\{q_4\}$	$\{q_4\}$	$\emptyset$

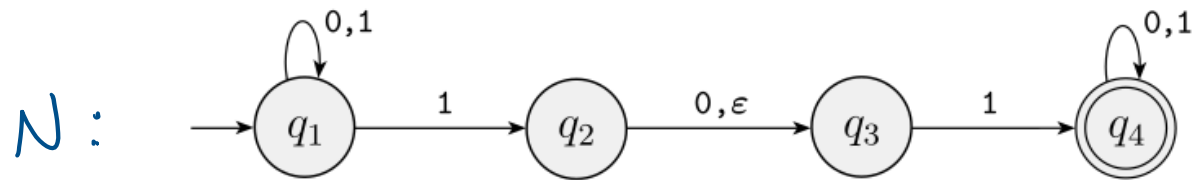
4. q_1 é o estado inicial, e
5. $F = \{q_4\}$.

$$\hat{\delta}(q_1, \epsilon) =$$

$$\hat{\delta}(q_1, 0) =$$

$$\hat{\delta}(q_1, 01) =$$

Exemplo: linguagem do AFN



$$L(N) = \{w \in \{0,1\}^* : w \text{ contém } 11 \text{ ou } 101 \text{ como subcadeia}\}$$

$$q_1 \in \hat{\delta}(q_1, w) \Leftrightarrow w \in \Sigma^*$$

$$q_2 \in \hat{\delta}(q_1, w) \Leftrightarrow w = \alpha 1, \text{ com } \alpha \in \Sigma^*$$

$$q_3 \in \hat{\delta}(q_1, w) \Leftrightarrow w = \alpha 1 \text{ ou } w = \alpha 10, \text{ com } \alpha \in \Sigma^*$$

$$q_4 \in \hat{\delta}(q_1, w) \Leftrightarrow w = \alpha 101\beta \text{ ou } w = \alpha 11\beta, \text{ com } \alpha, \beta \in \Sigma^*$$

Exemplos de AFN e suas linguagens

Outros exemplos no vídeo da profa. Carla Negri Lintzmayer em:

<https://youtu.be/mlaF5aT2rnE>

Continuação: Operações regulares

DEFINIÇÃO 1.23

Sejam A e B linguagens. Definimos as operações regulares *união*, *concatenação*, e *estrela* da seguinte forma.

- **União:** $A \cup B = \{x \mid x \in A \text{ ou } x \in B\}$.
- **Concatenação:** $A \circ B = \{xy \mid x \in A \text{ e } y \in B\}$.
- **Estrela:** $A^* = \{x_1 x_2 \dots x_k \mid k \geq 0 \text{ e cada } x_i \in A\}$.

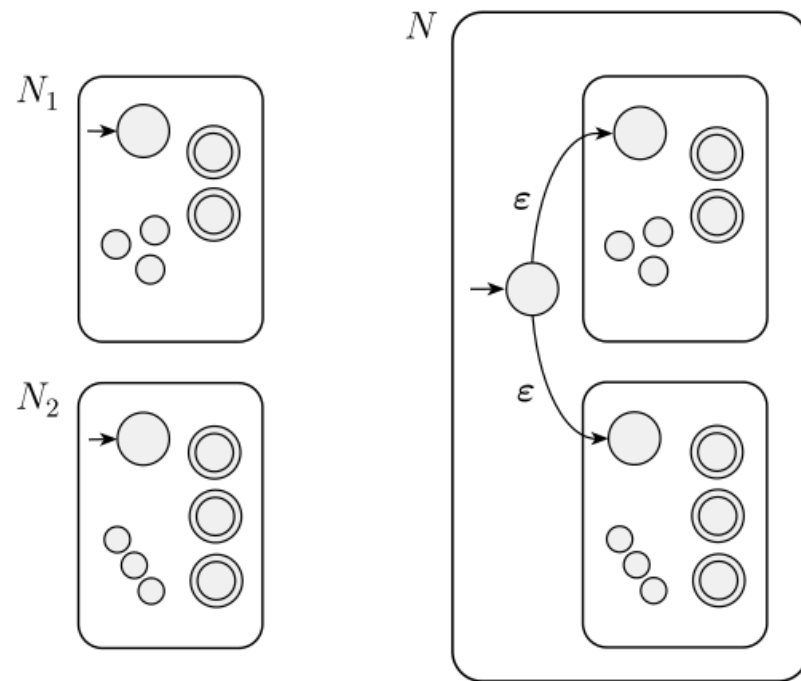
Continuação: Operações regulares

TEOREMA 1.45

A classe de linguagens regulares é fechada sob a operação de união.

IDÉIA DA PROVA Temos as linguagens regulares A_1 e A_2 e desejamos provar que $A_1 \cup A_2$ é regular. A idéia é tomar os dois AFNs, N_1 e N_2 para A_1 e A_2 , e combiná-los em um novo AFN, N .

A máquina N tem que aceitar sua entrada se N_1 ou N_2 aceita essa entrada. A nova máquina tem um novo estado inicial que ramifica para os estados iniciais das máquinas anteriores com setas ϵ . Dessa maneira a nova máquina não-deterministicamente adivinha qual das duas máquinas aceita a entrada. Se uma delas aceita a entrada, N também a aceitará.



Continuação: Operações regulares

TEOREMA 1.45

A classe de linguagens regulares é fechada sob a operação de união.

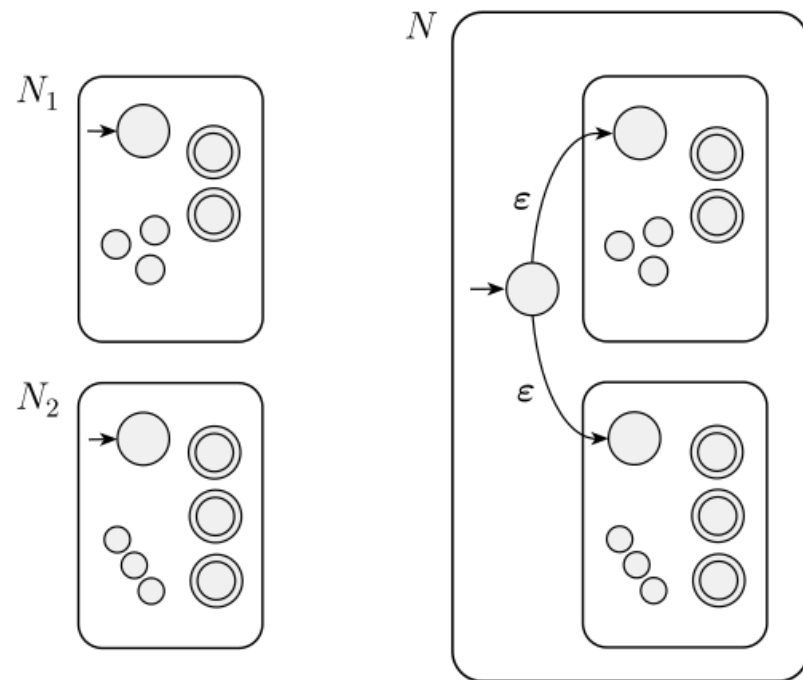
PROVA

Suponha que $N_1 = (Q_1, \Sigma, \delta_1, q_1, F_1)$ reconheça A_1 , e
 $N_2 = (Q_2, \Sigma, \delta_2, q_2, F_2)$ reconheça A_2 .

Construa $N = (Q, \Sigma, \delta, q_0, F)$ para reconhecer $A_1 \cup A_2$.

1. $Q = \{q_0\} \cup Q_1 \cup Q_2$.
Os estados de N são todos os estados de N_1 e N_2 , com a adição de um novo estado inicial q_0 .
2. O estado q_0 é o estado inicial de N .
3. Os estados de aceitação $F = F_1 \cup F_2$.
Os estados de aceitação de N são todos os estados de aceitação de N_1 e N_2 .
Dessa forma N aceita se N_1 aceita ou N_2 aceita.
4. Defina δ de modo que para qualquer $q \in Q$ e qualquer $a \in \Sigma_\epsilon$,

$$\delta(q, a) = \begin{cases} \delta_1(q, a) & q \in Q_1 \\ \delta_2(q, a) & q \in Q_2 \\ \{q_1, q_2\} & q = q_0 \text{ e } a = \epsilon \\ \emptyset & q = q_0 \text{ e } a \neq \epsilon. \end{cases}$$

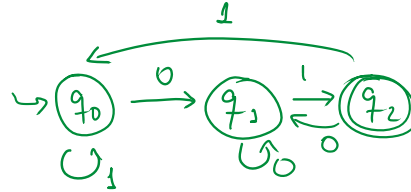


Exemplo: união

$L_1 = \{w \in \{0,1\}^* : w \text{ possui um número par de 0's}\}$

$L_2 = \{w \in \{0,1\}^* : w \text{ termina em } 01\}$

Pede-se: encontrar um AFD para reconhecer $L_1 \cup L_2 = \{w \in \{0,1\}^* : w \text{ termina em } 01 \text{ ou possui n}^\circ \text{ par de 0's}\}$

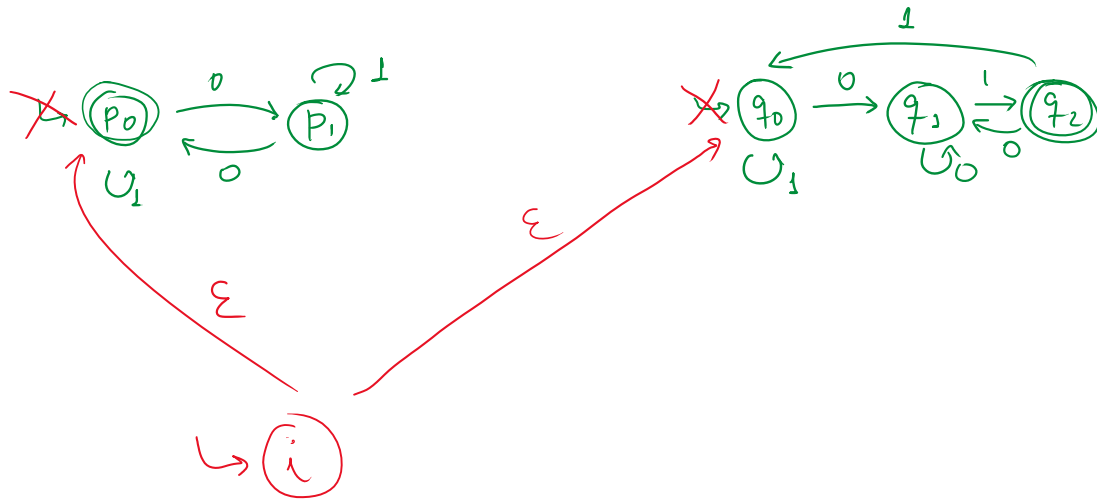


Exemplo: união

$L_1 = \{w \in \{0,1\}^* : w \text{ possui um número par de 0's}\}$

$L_2 = \{w \in \{0,1\}^* : w \text{ termina em } 01\}$

Pede-se: encontrar um AFD para reconhecer $L_1 \cup L_2 = \{w \in \{0,1\}^* : w \text{ termina em } 01 \text{ ou possui n}^\circ \text{ par de 0's}\}$

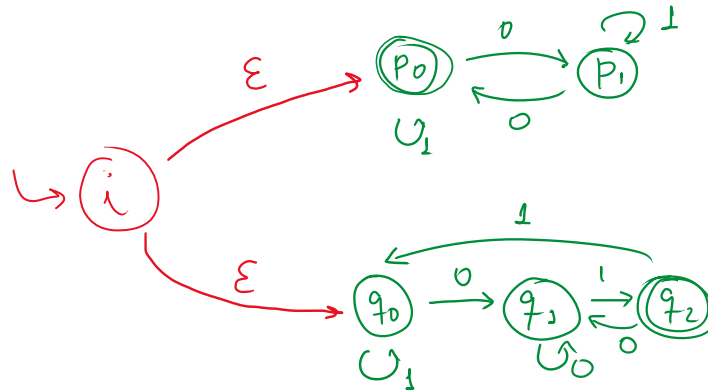


Exemplo: união

$L_1 = \{w \in \{0,1\}^* : w \text{ possui um número par de 0's}\}$

$L_2 = \{w \in \{0,1\}^* : w \text{ termina em } 01\}$

Pede-se: encontrar um AFD para reconhecer $L_1 \cup L_2 = \{w \in \{0,1\}^* : w \text{ termina em } 01 \text{ ou possui n}^\circ \text{ par de 0's}\}$



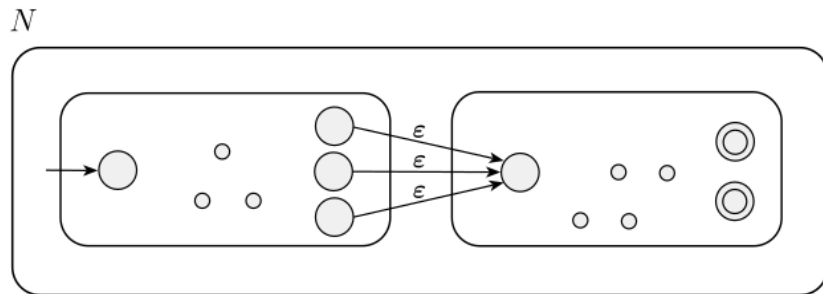
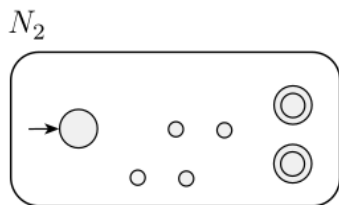
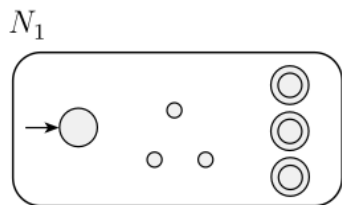
Operações regulares: concatenação

TEOREMA 1.47

A classe de linguagens regulares é fechada sob a operação de concatenação.

Atribua ao estado inicial de N o estado inicial de N_1 . Os estados de aceitação de N_1 têm setas ε adicionais que não-deterministicamente permitem ramificar para N_2 sempre que N_1 está num estado de aceitação, significando que ele encontrou uma parte inicial da entrada que constitui uma cadeia em A_1 . Os estados de aceitação de N são somente os estados de aceitação de N_2 . Por conseguinte, ele aceita quando a entrada pode ser dividida em duas partes, a primeira aceita por N_1 e a segunda por N_2 . Podemos pensar em N como não-deterministicamente adivinhando onde fazer a divisão.

Concatenação: $A \circ B = \{xy \mid x \in A \text{ e } y \in B\}$.



Operações regulares: concatenação

TEOREMA 1.47

A classe de linguagens regulares é fechada sob a operação de concatenação.

PROVA

Suponha que $N_1 = (Q_1, \Sigma, \delta_1, q_1, F_1)$ reconheça A_1 , e
 $N_2 = (Q_2, \Sigma, \delta_2, q_2, F_2)$ reconheça A_2 .

Construa $N = (Q, \Sigma, \delta, q_1, F_2)$ para reconhecer $A_1 \circ A_2$.

1. $Q = Q_1 \cup Q_2$.

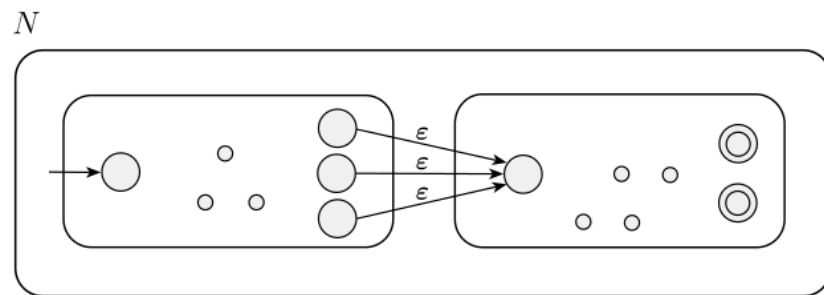
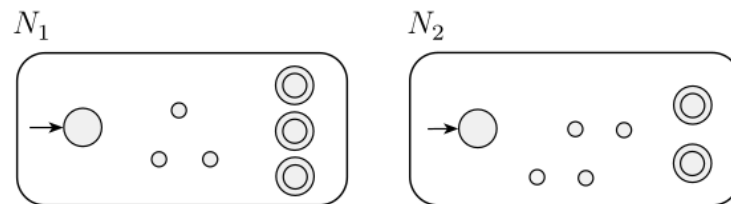
Os estados de N são todos os estados de N_1 e N_2 .

2. O estado q_1 é o mesmo que o estado inicial de N_1 .

3. Os estados de aceitação F_2 são os mesmos que os estados de aceitação de N_2 .

4. Defina δ de modo que para qualquer $q \in Q$ e qualquer $a \in \Sigma_\epsilon$,

$$\delta(q, a) = \begin{cases} \delta_1(q, a) & q \in Q_1 \text{ e } q \notin F_1 \\ \delta_1(q, a) & q \in F_1 \text{ e } a \neq \epsilon \\ \delta_1(q, a) \cup \{q_2\} & q \in F_1 \text{ e } a = \epsilon \\ \delta_2(q, a) & q \in Q_2. \end{cases}$$



Usando concatenação para construir AFNs de linguagens complexas

$$L = \{xa^m ba^n : x \in \{a, b\}^*, m + n \text{ é par e } x \text{ tem uma quantidade par de } a\text{'s}\}$$

Passo 1: quebrar a cadeia em subpartes independentes

Passo 2: construir um AFN para cada parte

Passo 3: conectar cada AFN com uma transição vazia (ϵ)

Passo 4: converter o AFN em AFD

Usando concatenação para construir AFNs de linguagens complexas

$$L = \{xa^m ba^n : x \in \{a, b\}^*, m + n \text{ é par e } x \text{ tem uma quantidade par de } a\text{'s}\}$$

x

a^m

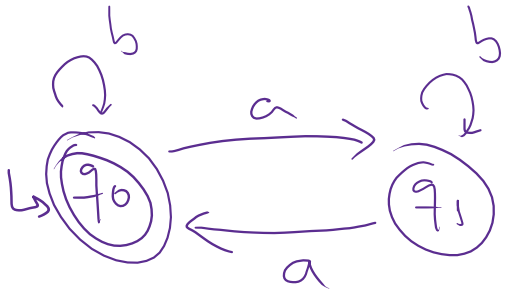
b

a^n

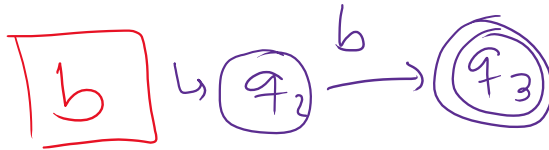
Usando concatenação para construir AFNs de linguagens complexas

$L = \{xa^m ba^n : x \in \{a, b\}^*, m + n \text{ é par e } x \text{ tem uma quantidade par de } a\text{'s}\}$

x



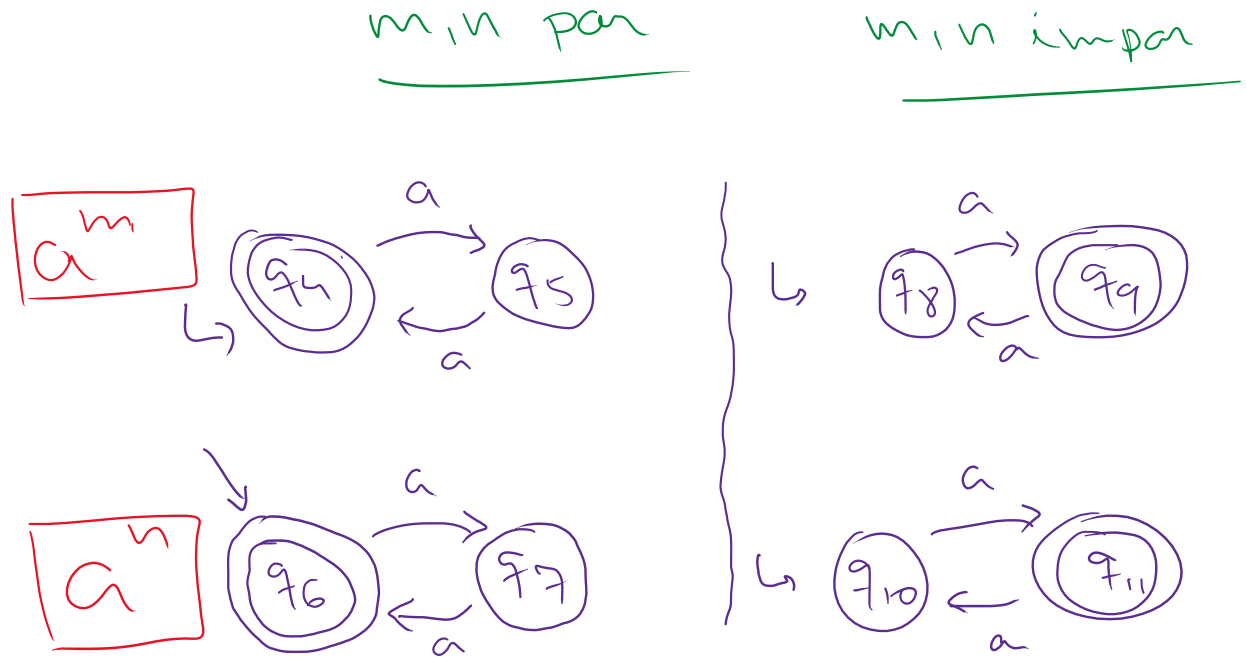
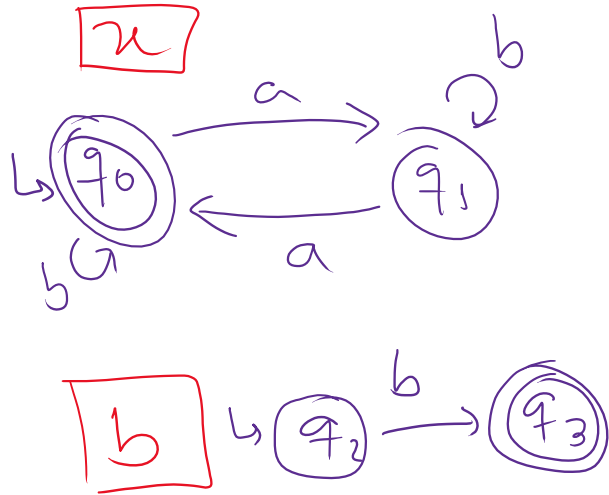
a^m



a^n

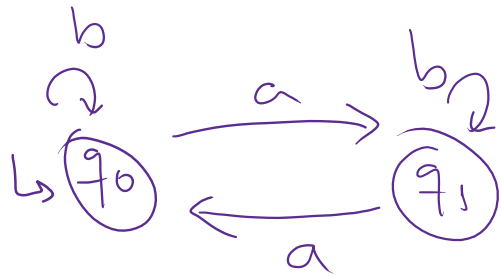
Usando concatenação para construir AFNs de linguagens complexas

$L = \{xa^m ba^n : x \in \{a, b\}^*, m + n \text{ é par e } x \text{ tem uma quantidade par de } a\text{'s}\}$

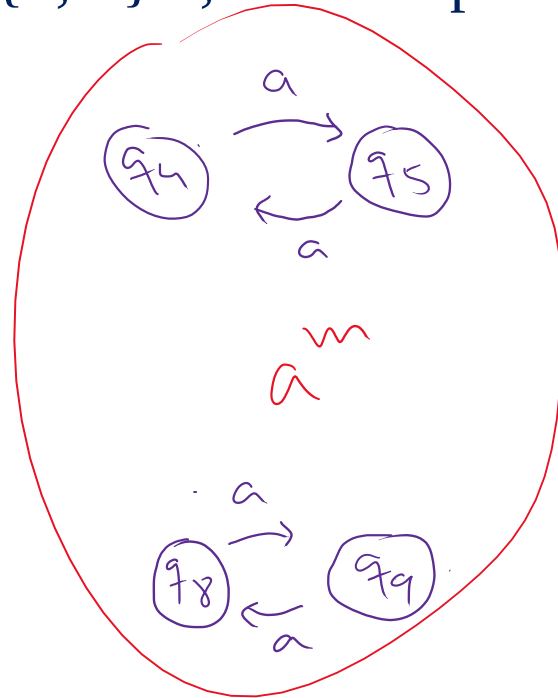


Usando concatenação para construir AFNs de linguagens complexas

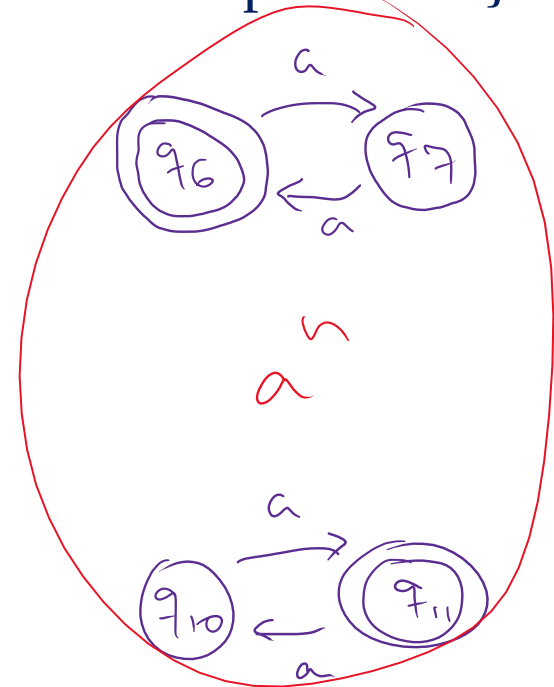
$L = \{xa^m ba^n : x \in \{a, b\}^*, m + n \text{ é par e } x \text{ tem uma quantidade par de } a\text{'s}\}$



x

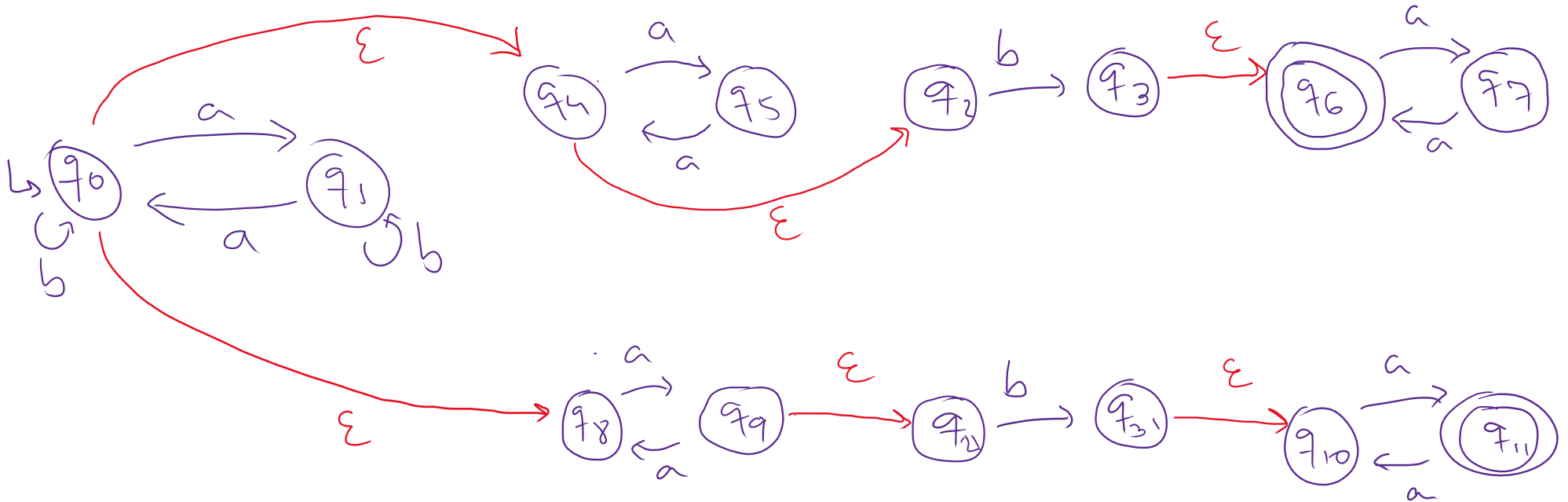


$\left. \begin{array}{c} \text{cópia} \end{array} \right\}$



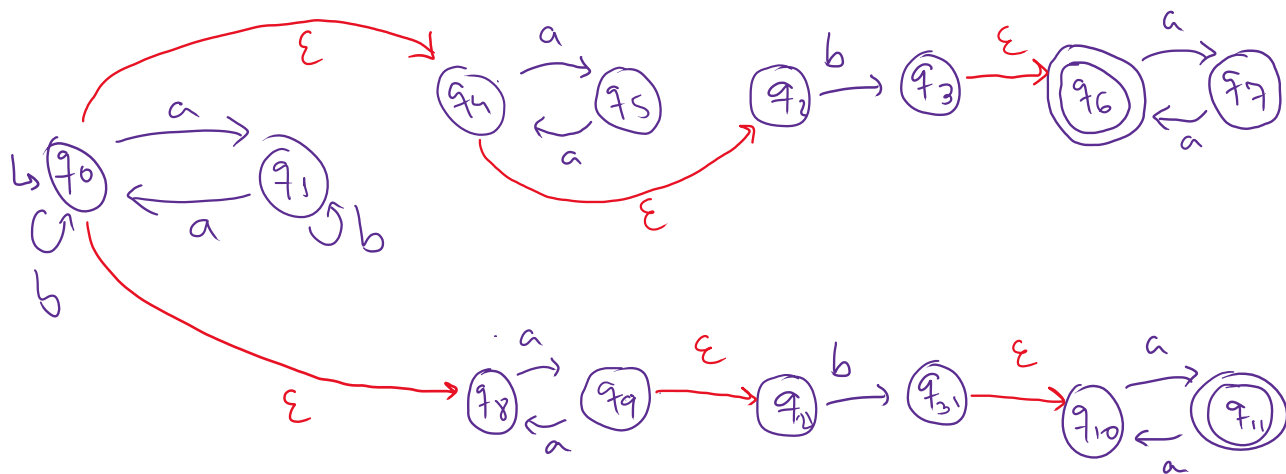
Usando concatenação para construir AFNs de linguagens complexas

$L = \{xa^m ba^n : x \in \{a, b\}^*, m + n \text{ é par e } x \text{ tem uma quantidade par de } a\text{'s}\}$



AFNs como facilitadores na construção de reconhecedores

$L = \{xa^m b a^n : x \in \{a, b\}^*, m + n \text{ é par e } x \text{ tem uma quantidade par de } a\text{'s}\}$



Relembrando: Quais são os fechos-vazios de cada estado?

$$E(R) = \{q \mid q \text{ pode ser atingido a partir de } R \text{ viajando-se ao longo de } 0 \text{ ou mais setas } \epsilon\}.$$

Operações regulares: estrela

TEOREMA 1.49

A classe de linguagens regulares é fechada sob a operação estrela.

PROVA Suponha que $N_1 = (Q_1, \Sigma, \delta_1, q_1, F_1)$ reconheça A_1 .
Construa $N = (Q, \Sigma, \delta, q_0, F)$ para reconhecer A_1^* .

1. $Q = \{q_0\} \cup Q_1$.

Os estados de N são os estados de N_1 mais um novo estado inicial.

2. O estado q_0 é o novo estado inicial.

3. $F = \{q_0\} \cup F_1$.

Os estados de aceitação são os antigos estados de aceitação mais o novo estado inicial.

4. Defina δ de modo que para qualquer $q \in Q$ e qualquer $a \in \Sigma_\epsilon$,

$$\delta(q, a) = \begin{cases} \delta_1(q, a) & q \in Q_1 \text{ e } q \notin F_1 \\ \delta_1(q, a) & q \in F_1 \text{ e } a \neq \epsilon \\ \delta_1(q, a) \cup \{q_1\} & q \in F_1 \text{ e } a = \epsilon \\ \{q_1\} & q = q_0 \text{ e } a = \epsilon \\ \emptyset & q = q_0 \text{ e } a \neq \epsilon. \end{cases}$$

Estrela: $A^* = \{x_1x_2 \dots x_k \mid k \geq 0 \text{ e cada } x_i \in A\}$.

